Práctica 1
Búsqueda local y trayectorias

Problema del Viajante de Comercio (TSP)

Algoritmos implementados: Greedy · Greedy aleatorio ·
Búsqueda local del primer mejor con *Don't Look Bits* ·
Búsqueda tabú con memoria a corto y largo plazo y oscilación estratégica

Alumno 1	Marco Carofiglio Pajares — 77382906G
Alumno 2	José María Valero Palomares — 78280274G
Guion	Práctica 1 — Búsqueda local y trayectorias
Clase	1
Grupo	2

Índice

Índice **2**
1. Descripción del problema **3**
1.1 Planteamiento 3
1.2 Esquema de representación de soluciones 3
1.3 Descripción del código 3
1.4 Función objetivo 4
1.5 Operadores y estructuras utilizados **4**
2. Descripción en pseudocódigo **6**
3. Experimentos y análisis de resultados **9**
3.1 Parámetros 9
3.2 Resultados 10
3.3 Análisis del comportamiento de cada algoritmo 15
3.4 Tiempos, desviaciones e influencia del tamaño 20
3.5 Análisis de la mejor solución obtenida 22
3.6 Ficheros de registro (log) **23**
4. Referencias bibliográficas **24**

1. Descripción del problema

1.1 Planteamiento

La empresa de transporte UJACAR necesita calcular cada día la ruta más corta de su camión de reparto. El problema se modela como un **Problema del Viajante de Comercio simétrico** (*Travelling Salesman Problem*, TSP): dado un conjunto $C = \{c_1, \ldots, c_n\}$ de n ciudades y una matriz de distancias $D = (d_{ij})$, se busca el circuito hamiltoniano de longitud mínima, es decir, un recorrido que visite cada ciudad exactamente una vez y regrese a la ciudad de partida.

El TSP es NP-duro: el espacio de búsqueda tiene $(n-1)!/2$ recorridos distintos, lo que hace inviable la enumeración incluso para tamaños moderados y justifica el uso de heurísticas y metaheurísticas. Cada ciudad viene dada por sus coordenadas en el plano y la distancia entre dos ciudades es la euclídea:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (1)$$

Al ser simétrica ($d_{ij} = d_{ji}$), un recorrido y su inverso tienen el mismo coste, y cualquier rotación de la permutación representa el mismo circuito. Se utilizan cinco instancias de tamaño creciente: **ch130** (130 ciudades), **pr144** (144), **a280** (280), **u1060** (1 060) y **d18512** (18 512), cuyos óptimos conocidos se recogen en la Tabla 2.

1.2 Esquema de representación de soluciones

Una solución se representa mediante una **permutación** $S = (s_1, s_2, \ldots, s_n)$ de los índices de las ciudades (numerados de 0 a $n-1$), donde s_k es la ciudad visitada en la posición k del recorrido. El circuito se cierra con el arco (s_n, s_1). Esta representación garantiza que toda solución es factible (cada ciudad aparece exactamente una vez).

Para que los operadores trabajen en tiempo constante se mantiene, junto a la permutación, un **vector inverso de posiciones** *pos*, tal que $pos[c] = k \Leftrightarrow s_k = c$. Ambos vectores se actualizan conjuntamente tras cada movimiento.

1.3 Descripción del código

La práctica está implementada en Java y se organiza en cuatro bloques funcionales:

- **Configurador.** Lee un fichero de parámetros con pares *clave* = *valor* que determina qué instancias se procesan, qué algoritmos se ejecutan, con qué semillas, qué algoritmos vuelcan su traza a fichero o a consola y los parámetros de la búsqueda tabú (iteraciones, tenencia, estancamiento y oscilación).
- **Lector de instancias.** Interpreta la cabecera (nombre, dimensión, tipo de distancia) y la sección de coordenadas, y construye una única vez la matriz completa de distancias $n \times n$ según la ecuación (1). Todos los algoritmos consultan esta matriz, por lo que cada evaluación de un arco cuesta $O(1)$.
- **Algoritmos.** Un módulo por algoritmo: el Greedy, el Greedy aleatorio (que especializa al anterior y reutiliza su ordenación de ciudades), la Búsqueda local y la Búsqueda tabú. Cada algoritmo recibe la matriz de distancias (y, en su caso, la solución inicial, la semilla y los parámetros) y expone la mejor solución, su coste y las estadísticas de la ejecución (iteraciones, reinicios, etc.).

- **Registro de ejecución.** Un componente de *log* acumula en memoria la traza de cada ejecución —cabecera con fecha, instancia, algoritmo y semilla; cada movimiento aplicado con su incremento de coste; los reinicios y el resultado final— y la vuelca a un fichero de texto independiente al terminar. Escribir al final, y no línea a línea, evita que la entrada/salida distorsione los tiempos medidos.

El programa principal recorre las instancias y, para cada una, los algoritmos y las semillas configurados. Para una misma semilla, la solución del Greedy aleatorio se calcula una sola vez y se reutiliza como **solución de partida común** de la Búsqueda local y de la Búsqueda tabú, de forma que la comparación entre ambas es pareada y no está contaminada por diferencias en el punto de inicio. El cronómetro mide únicamente el tiempo del algoritmo evaluado (sin la lectura de la instancia ni la construcción de la solución inicial).

1.4 Función objetivo

Se minimiza la longitud total del circuito:

$$\min C(S) = \Sigma_{k=1}^{n-1} d_{s_k s_{k+1}} + d_{s_n s_1} \quad (2)$$

El cálculo completo de (2) tiene coste $O(n)$ y solo se realiza al crear o reconstruir una solución. Durante la búsqueda, el coste se mantiene de forma incremental mediante la **factorización** del movimiento (apartado 1.5), que obtiene la variación Δ en $O(1)$; el coste nuevo es $C(S') = C(S) + \Delta$.

1.5 Operadores y estructuras utilizados

Operador de vecindario: intercambio de dos ciudades

Los algoritmos de trayectoria utilizan el operador de **intercambio** (*2opt*): dadas dos posiciones $i \neq j$, se intercambian las ciudades $a = s_i$ y $b = s_j$. El incremento de coste se calcula sin evaluar el recorrido completo, considerando solo los arcos que cambian.

Un movimiento se considera de mejora solo si $\Delta < -\varepsilon$, con $\varepsilon = 10^{-10}$, para no aceptar «mejoras» debidas exclusivamente al error de redondeo en coma flotante, que harían ciclar la búsqueda local entre soluciones de igual coste.

Máscara *Don't Look Bits* (DLB)

Se mantiene un bit por ciudad. Si el bit de una ciudad vale 1, se omite la exploración de sus movimientos, porque en la última visita no produjo ninguna mejora. Cuando se aplica un intercambio, los bits de las dos ciudades implicadas vuelven a 0, ya que su vecindad ha cambiado. La búsqueda local alcanza un óptimo local respecto al operador cuando todos los bits valen 1. La máscara reduce mucho el número de evaluaciones, porque concentra el esfuerzo en las zonas del recorrido que acaban de modificarse.

Memorias de la búsqueda tabú

Ambas memorias comparten una única matriz $n \times n$ de enteros cortos, lo que aprovecha que los dos atributos que registran son pares no ordenados de ciudades:

- **Memoria a corto plazo (MCP), en el triángulo superior.** Tras intercambiar las ciudades *a* y *b*, la celda *(min(a,b), max(a,b))* recibe el valor de la **tenencia tabú**. Antes de cada movimiento se decrementan todas las celdas activas. Un intercambio entre *a* y

b es tabú mientras su celda sea positiva. Una lista auxiliar guarda las celdas activas para que el decremento no cueste $O(n^2)$. No se aplica criterio de aspiración.

- **Memoria a largo plazo (MLP), en el triángulo inferior.** Es una memoria de frecuencias: cada vez que se visita una solución (incluidas la inicial y las de los reinicios), se incrementa el contador de los *n* arcos que la forman. Por tanto, la MLP cuantifica con qué frecuencia ha formado parte cada arco de las soluciones recorridas.

Oscilación estratégica: generación de soluciones a partir de la MLP

Cuando la búsqueda se estanca, se construye una solución nueva con un esquema voraz guiado por la MLP. Se parte de una ciudad aleatoria y en cada paso se elige, entre las ciudades no visitadas, la unida a la actual por el arco **más frecuente** (*intensificación*, que explota las regiones prometedoras) o por el **menos frecuente** (*diversificación*, que conduce a regiones poco exploradas). Los empates se resuelven a favor de la ciudad más cercana. La elección entre intensificar y diversificar es aleatoria, con probabilidad de intensificar igual al parámetro de oscilación.

2. Descripción en pseudocódigo

Se presentan los algoritmos en el orden en que intervienen. Se emplea notación de vectores con índices modulares para el cierre del circuito; *D* denota la matriz de distancias y *n* el número de ciudades.

Algoritmo 1. Greedy (ordenación por centralidad)

```
Entrada: matriz D · Salida: permutación S
1  para cada ciudad i ∈ {0, …, n−1} hacer
2      suma[i] ← Σ_j D[i][j] — distancia total de i al resto
3  L ← ciudades ordenadas por suma[·] ascendente — las más «céntricas» primero
4  S ← L
5  devolver S, C(S)
```

Algoritmo 2. Greedy aleatorio

```
Entrada: matriz D, semilla, K = 5 · Salida: permutación S
1  L ← ciudades ordenadas por suma de distancias ascendente — como en el Algoritmo 1
2  inicializar el generador aleatorio con la semilla
3  S ← ()
4  mientras L no esté vacía hacer
5      k ← min(K, |L|)
6      i ← aleatorio en {0, …, k-1}  - una de las k ciudades más centricas que queden
7      c ← L[i]; eliminar L[i] de L
8      añadir c al final de S;
9  devolver S, C(S)
```

El greedy aleatorio parte de la misma lista L que el greedy determinista, ordenada por suma de distancias ascendente. En cada paso no toma siempre la primera ciudad de L, sino una elegida al azar entre las K = 5 primeras que quedan sin visitar, y la elimina de la lista. Así se obtiene un recorrido parecido al del greedy, pero con variaciones locales en el orden que dependen de la semilla. Al no haber lista de visitadas aparte, eliminar la ciudad elegida de L basta para que no se repita.

Algoritmo 3. Factorización del intercambio — Δ(S, i, j)

```
Entrada: solución S, posiciones i ≠ j · Salida: variación de coste Δ
1  a ← S[i]; b ← S[j]
2  p_i ← S[i−1]; q_i ← S[i+1]; p_j ← S[j−1]; q_j ← S[j+1] — índices módulo n
3  si j = i+1 (mod n) entonces devolver D[p_i][b] + D[a][q_j] − D[p_i][a] − D[b][q_j]
4  si i = j+1 (mod n) entonces devolver D[p_j][a] + D[b][q_i] − D[p_j][b] − D[a][q_i]
5  devolver D[p_i][b] + D[b][q_i] + D[p_j][a] + D[a][q_j] − D[p_i][a] − D[a][q_i] − D[p_j][b] − D[b][q_j]
```

Algoritmo 4. Búsqueda local del primer mejor con DLB

```
Entrada: solución inicial S₀, D, máximo de iteraciones M = 10 000 · Salida:
óptimo local S
 1  S ← S₀; coste ← C(S); calcular pos; DLB[·] ← 0; it ← 0
 2  c ← 0; sinMejora ← 0
 3  mientras sinMejora < n y it < M hacer
 4      si DLB[c] = 1 entonces sinMejora ← sinMejora + 1
 5      si no
 6          i ← pos[c]; mejorado ← falso
 7          para desp ← 1 hasta n−1 y mientras no mejorado hacer
 8              j ← (i + desp) mod n; δ ← Δ(S, i, j)
 9              si δ < −ε entonces — primer mejor: se acepta sin explorar
                el resto
10                  intercambiar S[i] y S[j]; actualizar pos; coste ← coste
                    + δ; it ← it + 1
11                  DLB[S[i]] ← 0; DLB[S[j]] ← 0; mejorado ← cierto
12          si mejorado entonces sinMejora ← 0
13          si no DLB[c] ← 1; sinMejora ← sinMejora + 1
14      c ← (c + 1) mod n
15  devolver S, coste
```

La condición *sinMejora* ≥ *n* equivale a haber dado una vuelta completa sobre las ciudades sin encontrar ninguna mejora, es decir, a tener toda la máscara DLB a 1. La exploración de cada ciudad comienza en su posición y avanza circularmente, de modo que primero se prueban los vecinos de posición más cercanos.

Algoritmo 5. Búsqueda tabú — esquema general

```
Entrada: S₀, D, semilla, M = 10 000, tenencia T = 10, estancamiento e = 5
%, oscilación o = 50 % · Salida: mejor global S*
 1  S ← S₀; S* ← S; MCP ← 0; MLP ← 0; DLB ← 0; it ← 0
 2  ActualizarMLP(S)
 3  mientras it < M hacer
 4      convergido ← FaseLS(S) — Algoritmo 4; cada movimiento suma a la
        MLP y a it
 5      ActualizarMejorGlobal(S)
 6      si no convergido entonces terminar — presupuesto agotado antes del
        óptimo local
 7      S_mom ← S — mejor del momento actual
 8      estancado ← FaseTS(S, S_mom) — Algoritmo 6
 9      si no estancado entonces terminar
10      intensificar ← (aleatorio[0,1) < o)
11      S ← ConstruirDesdeMLP(intensificar) — Algoritmo 7
12      MCP ← 0; DLB ← 0; S_mom ← S; ActualizarMejorGlobal(S);
        ActualizarMLP(S) — la MLP no se reinicia
13  devolver S*
```

Algoritmo 6. FaseTS — búsqueda tabú hasta estancamiento

```
Entrada: S, S_mom · Salida: cierto si termina por estancamiento
 1  umbral ← max(1, ⌊M · e⌋) — 500 movimientos con los parámetros usados
 2  empeoramientos ← 0; c ← 0
 3  mientras it < M y empeoramientos < umbral hacer
 4      i ← pos[c]; j* ← nulo; δ* ← +∞
 5      para cada posición j ≠ i hacer
 6          si MCP[c, S[j]] = 0 y Δ(S, i, j) < δ* entonces j* ← j; δ* ←
            Δ(S, i, j)
 7      si j* = nulo entonces DecrementarMCP(); c ← (c+1) mod n; continuar
 8      b ← S[j*]; DecrementarMCP()
 9      intercambiar S[i] y S[j*]; MCP[c, b] ← T; ActualizarMLP(S)
10      coste ← coste + δ*; it ← it + 1 — se acepta aunque δ* > 0
11      si C(S) < C(S_mom) − ε entonces S_mom ← S; empeoramientos ← 0
12      si no empeoramientos ← empeoramientos + 1
13      ActualizarMejorGlobal(S); c ← (c+1) mod n
14  devolver empeoramientos ≥ umbral
```

En cada iteración se fija una ciudad (recorridas cíclicamente) y se aplica el **mejor intercambio no tabú** de esa ciudad, aunque empeore la solución. Es esta aceptación de movimientos de empeoramiento la que permite salir del óptimo local. La prohibición del par recién intercambiado durante *T* iteraciones impide deshacer inmediatamente el movimiento y caer en ciclos cortos.

Algoritmo 7. ConstruirDesdeMLP — oscilación estratégica

```
Entrada: MLP, D, intensificar ∈ {cierto, falso} · Salida: nueva
permutación S
 1  c ← ciudad aleatoria; S ← (c); visitada[c] ← cierto
 2  para k ← 1 hasta n−1 hacer
 3      si intensificar entonces x ← argmax_{x no visitada} MLP[c, x]
 4      si no x ← argmin_{x no visitada} MLP[c, x]
 5      — en caso de empate de frecuencia, se elige la x con menor
        D[c][x]
 6      añadir x a S; visitada[x] ← cierto; c ← x
 7  devolver S
```

Las operaciones auxiliares son inmediatas: *ActualizarMLP(S)* incrementa la frecuencia de los *n* arcos (s_k, s_{k+1}) de la solución; *DecrementarMCP()* resta una unidad a las celdas tabú activas y retira las que llegan a cero; *ActualizarMejorGlobal(S)* sustituye S^* si $C(S) < C(S^*) - \varepsilon$. La solución devuelta es siempre la mejor global, no la última visitada.

3. Experimentos y análisis de resultados

3.1 Parámetros

Parámetro	Valor	Significado
Semillas	12345678, 23456781, 34567812	Tres ejecuciones independientes por algoritmo estocástico e instancia.
K (Greedy aleatorio)	5	Tamaño de la lista de candidatos de la ciudad inicial y de cada paso
Iteraciones máximas	10 000	Movimientos aplicados, tanto en la Búsqueda local como en la Búsqueda tabú (fase LS + fase TS)
Tenencia tabú T	10	Iteraciones durante las que un par intercambiado permanece tabú
Estancamiento e	5 % (500 movimientos)	Movimientos consecutivos sin mejorar al mejor del momento que provocan el reinicio
Oscilación o	50 %	Probabilidad de reiniciar por intensificación (en otro caso, diversificación)

Como solución inicial de la Búsqueda local y de la Búsqueda tabú se usa, para cada semilla, la misma solución del Greedy aleatorio, que se recoge también como resultado del propio Greedy aleatorio. El Greedy es determinista y se ejecuta una sola vez por instancia. Los experimentos se ejecutaron en un equipo con procesador AMD Ryzen 5 2600 (6 núcleos, 12 hilos) y 16 GB de RAM, bajo Windows 11 y Java 25 (OpenJDK).

3.2 Resultados

	A280		CH130		D18512		PR144		U1060		MEDIA	
	Media	**Time**	***Media***	**Time**	***Media***	**Time**	***Media***	**Time**	***Media***	**Time**	***C***	**Time**
Greedy	1074,75%	17,89	561,17%	0,62	7829,12%	425,75	994,64%	0,26	2430,22%	2,17	2577,98%	89,34
Greedy Aleatorio	1048,31%	5,70	623,05%	0,66	7862,20%	425,89	1047,05%	0,18	2397,10%	1,96	2595,54%	86,88
Búsqueda Local	168,35%	80,49	91,85%	6,95	5764,98%	125,14	201,13%	4,62	375,29%	309,99	1320,32%	105,44
TABU	37,28%	526,74	20,76%	150,36	5764,98%	7604,12	51,73%	87,09	139,81%	620,33	1202,91%	1797,73
MA												

«Tabla 1»

	A280		CH130		D18512		PR144		U1060	
Greedy	**Tamaño**	280	**Tamaño**	130	**Tamaño**	18512	**Tamaño**	144	**Tamaño**	1060
	Mínimo encontrado	2579	**Mínimo encontrado**	6110	**Mínimo encontrado**	645238	**Mínimo encontrado**	58537	**Mínimo encontrado**	224094
	Sol	**Time**	***Sol***	**Time**	***Sol***	**Time**	***Sol***	**Time**	***Sol***	**Time**
Ejecución 1	30296,82	17,89	40397,47	0,62	51161687,81	425,75	640770,88	0,26	5670064,49	2,17
Media	1074,75%	17,89	561,17%	0,62	7829,12%	425,75	994,64%	0,26	2430,22%	2,17
Desv. típica	—	—	—	—	—	—	—	—	—	—

«Tabla 2»

	A280		CH130		D18512		PR144		U1060	
Greedy Aleatorio	Tamaño	280	Tamaño	130	Tamaño	18512	Tamaño	144	Tamaño	1060
	Mínimo encontrado	2579	Mínimo encontrado	6110	Mínimo encontrado	645238	Mínimo encontrado	58537	Mínimo encontrado	224094
	Sol	Time	*Sol*	Time	*Sol*	Time	*Sol*	Time	*Sol*	Time
Ejecución 1	30277,62	7,60	44669,51	0,66	51435366,98	423,01	676433,85	0,20	5496158,64	1,99
Ejecución 2	29955,80	6,10	45344,45	0,69	51155822,34	417,65	645750,02	0,16	5741888,67	1,93
Ejecución 3	28611,37	3,41	42520,63	0,62	51534243,54	437,03	692159,74	0,18	5549533,87	1,95
Media	1048,31%	5,70	623,05%	0,66	7862,20%	425,89	1047,05%	0,18	2397,10%	1,96
Desv. típica	0,34	2,12	0,24	0,03	0,30	10,01	0,40	0,02	0,58	0,03

«Tabla 3»

	A280		CH130		D18512		PR144		U1060	
Búsqueda Local	Tamaño	280	Tamaño	130	Tamaño	18512	Tamaño	144	Tamaño	1060
	Mínimo encontrado	2579	Mínimo encontrado	6110	Mínimo encontrado	645238	Mínimo encontrado	58537	Mínimo encontrado	224094
	Sol	Time	*Sol*	Time	*Sol*	Time	*Sol*	Time	*Sol*	Time
Ejecución 1	6444,26	127,63	12238,21	6,65	37875970,07	237,65	1698344,48	4,81	1033783,80	313,08
Ejecución 2	7568,36	70,72	11237,36	7,45	37672567,17	74,33	1834505,05	4,55	1088946,20	338,22
Ejecución 3	6749,93	43,11	11690,79	6,75	37980685,53	63,46	1755311,17	4,51	1072583,02	278,68
Media	168,35%	80,49	91,85%	6,95	5764,98%	125,14	201,13%	4,62	375,29%	309,99
Desv. típica	0,23	43,10	0,08	0,43	0,24	97,58	0,12	0,16	0,13	29,89

«Tabla 4»

	A280		CH130		D18512		PR144		U1060	
Búsqueda Tabú	**Tamaño**	280	**Tamaño**	130	**Tamaño**	18512	**Tamaño**	144	**Tamaño**	1060
	Mínimo encontrado	2579	**Mínimo encontrado**	6110	**Mínimo encontrado**	645238	**Mínimo encontrado**	58537	**Mínimo encontrado**	224094
	Sol	**Time**	***Sol***	**Time**	***Sol***	**Time**	***Sol***	**Time**	***Sol***	**Time**
Ejecución 1	3302,57	663,28	7339,62	168,40	37875970,07	7615,87	91463,99	89,51	3876111,71	610,78
Ejecución 2	3827,52	466,40	7567,95	145,38	37672567,17	7335,25	88052,31	86,90	6070304,29	662,51
Ejecución 3	3491,32	450,53	7228,37	137,30	37980685,53	7861,24	86931,44	84,86	6175204,99	587,71
Media	37,28%	526,74	20,76%	150,36	5764,98%	7604,12	51,73%	87,09	139,81%	620,33
Desv. típica	0,10	118,51	0,03	16,13	0,24	263,19	0,04	2,33	0,58	38,30

«Tabla 5»

3.3 Análisis del comportamiento de cada algoritmo

Greedy

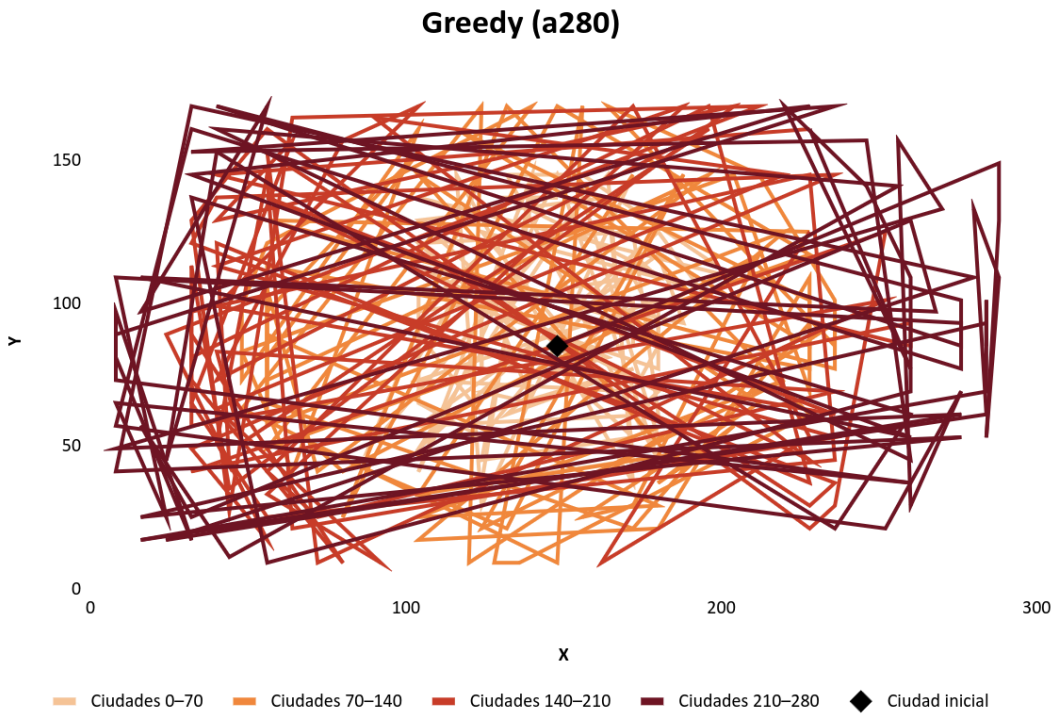
El Greedy ordena las ciudades según su distancia total al resto y las visita en ese orden. El criterio mide la centralidad de cada ciudad, pero no tiene en cuenta la proximidad entre ciudades consecutivas del recorrido. La Figura 1 (arriba) muestra la consecuencia. El recorrido empieza en la ciudad más céntrica, en (148, 85), y avanza por capas desde el centro hacia el borde: los arcos claros, que corresponden a las primeras ciudades, se concentran en el centro, y los oscuros, que corresponden a las últimas, en la periferia. Sin embargo, dentro de cada capa, dos ciudades igual de céntricas pueden estar en lados opuestos del centro geométrico, y el orden no las agrupa por proximidad. Por eso el recorrido cruza la región una y otra vez. El efecto se acentúa en el último tramo: las ciudades menos céntricas están repartidas por todo el contorno, y los arcos oscuros atraviesan la instancia casi de lado a lado. Su calidad es, por tanto, comparable a la de una permutación aleatoria: desviaciones del 561,17 % (ch130) al 7 829,12 % (d18512). Su utilidad es servir de cota de referencia y aportar la ordenación por centralidad sobre la que construye su recorrido el Greedy aleatorio.

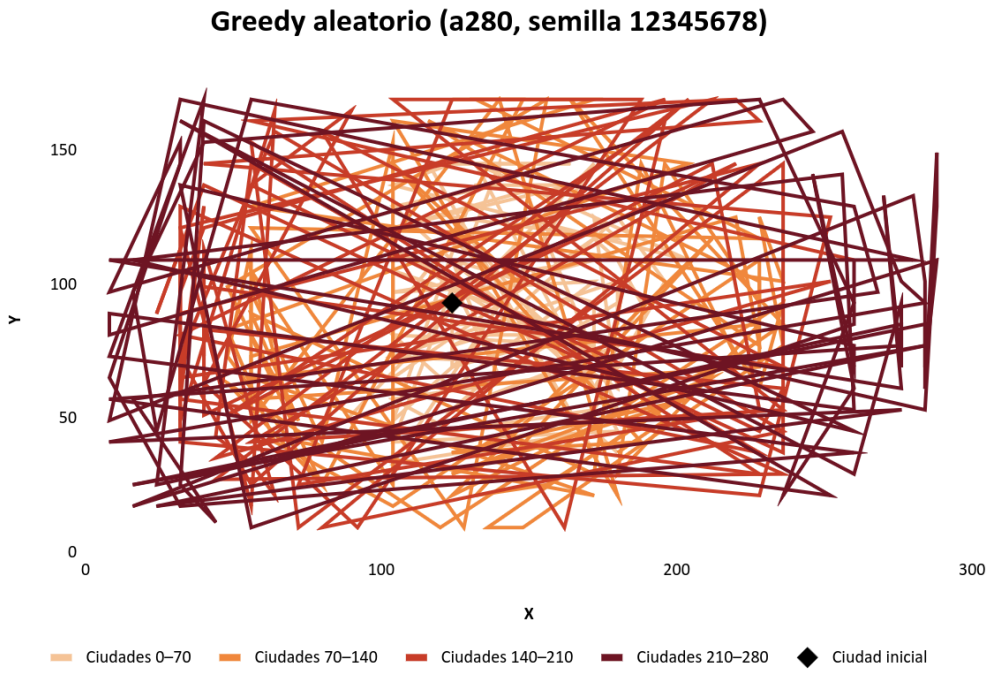
Greedy aleatorio

El Greedy aleatorio recorre la misma ordenación por centralidad que el Greedy, pero en cada paso elige al azar una de las cinco ciudades más céntricas que quedan sin visitar (Algoritmo 2). La aleatoriedad solo altera el orden de forma local: una ciudad puede adelantarse como mucho cuatro posiciones respecto al Greedy y rara vez se retrasa más de unas pocas. Por eso la Figura 1 (abajo) muestra prácticamente el mismo patrón que el Greedy: arcos claros en el centro, arcos oscuros hacia el borde y numerosos cruces en toda la región. La ciudad inicial es una de las cinco más céntricas, en (124, 93), muy cerca de la del Greedy. Como el criterio sigue sin tener en cuenta la proximidad entre ciudades consecutivas, permutar ciudades cercanas en la ordenación no acorta el recorrido de forma sistemática: en a280, la semilla 12345678 obtiene 30 277,62 frente a los 30 296,82 del Greedy.

Los costes son del mismo orden que los del Greedy: 595,92–642,14 % sobre el óptimo en ch130, 1 003,15–1 082,43 % en pr144, 1 009,40–1 074,01 % en a280, entre 24,53 y 25,62 veces el óptimo en u1060 y entre 79,28 y 79,87 veces en d18512. En a280 y u1060 el Greedy aleatorio mejora de media al Greedy en un 2,25 % y un 1,31 %, respectivamente; en ch130 y pr144 es peor, un 9,36 % y un 4,79 %, y en d18512 ambos son prácticamente equivalentes. La variabilidad entre semillas es reducida (coeficiente de variación del 0,38–3,52 %). Además, las semillas 12345678 y 23456781 producen la misma ciudad inicial en las cinco instancias, porque el primer número pseudoaleatorio de ambas cae en el mismo índice de la lista de candidatos. Las diferencias entre ellas proceden solo de los pasos posteriores.

Desde el punto de vista experimental, construir el recorrido por centralidad hace que las metaheurísticas partan de soluciones muy alejadas del óptimo. Es una prueba exigente de su capacidad de mejora, pero no representa el uso habitual, en el que se partiría de un vecino más cercano, cuyas desviaciones en instancias euclídeas rondan el 25 %





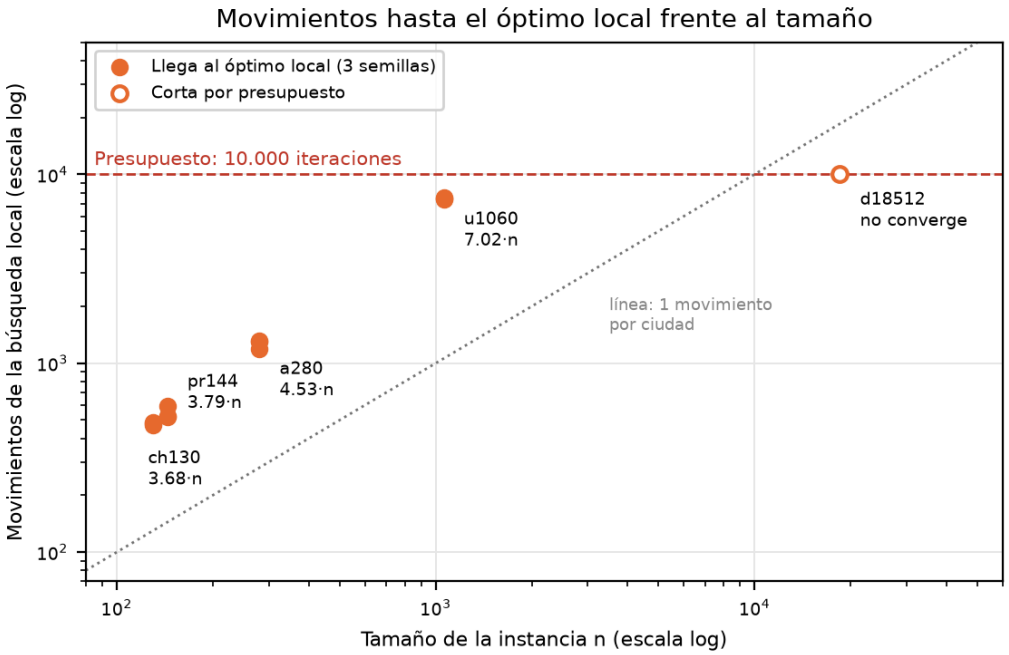
«Figura 1. Recorridos del Greedy (arriba) y del Greedy aleatorio con semilla 12345678 (abajo) en a280. El color indica el orden de visita, de claro (primeras ciudades) a oscuro (últimas); el rombo negro marca la ciudad inicial.»

Búsqueda local del primer mejor con DLB

La búsqueda local reduce el coste inicial entre un 71,59 % (pr144) y un 81,19 % (u1060) en las cuatro instancias en las que converge, y siempre termina en un óptimo local con toda la máscara DLB a 1. El número de movimientos necesarios crece aproximadamente de forma lineal con el tamaño: 3,68 n en ch130, 3,79 n en pr144, 4,53 n en a280 y 7,02 n en u1060. Que la razón crezca con n indica que, cuanto peor es la solución de partida en términos relativos, más movimientos de mejora encadena el descenso.

Aun así, los óptimos locales están muy lejos del óptimo global: entre el 91,85 % y el 375,29 % por encima. La causa principal es el **operador de intercambio**, poco adecuado para el TSP euclídeo. Un cruce entre dos arcos solo se deshace invirtiendo el tramo intermedio (movimiento 2-opt), algo que un intercambio de dos ciudades no consigue salvo en casos triviales. Por eso el descenso queda atrapado en recorridos con muchos cruces, y la superficie de búsqueda de este vecindario presenta muchos óptimos locales de mala calidad. Lo confirma la alta dispersión entre semillas (desviación típica del 2,66–8,40 % del coste medio): partiendo de soluciones iniciales de coste similar, se llega a óptimos locales bastante distintos.

En d18512 la búsqueda local **agota las 10 000 iteraciones** sin llegar al óptimo local. Con 18512 ciudades, el presupuesto no alcanza ni para un movimiento por ciudad, y la mejora es solo del 26,34 %. La mejora es casi lineal en el número de movimientos, porque en una solución tan mala casi cualquier ciudad tiene un intercambio de mejora entre sus primeros vecinos de posición, y el criterio del primer mejor lo encuentra casi de inmediato. Esto explica también su tiempo, sorprendentemente bajo (125,14 ms de media).



«Figura 2. Movimientos realizados por la búsqueda local hasta alcanzar el óptimo local, frente al tamaño de la instancia (escalas logarítmicas, tres semillas por instancia). La línea punteada indica un movimiento por ciudad y la discontinua roja, el presupuesto de 10.000 iteraciones. En d18512 la búsqueda se detiene por presupuesto antes de converger.»

Búsqueda tabú

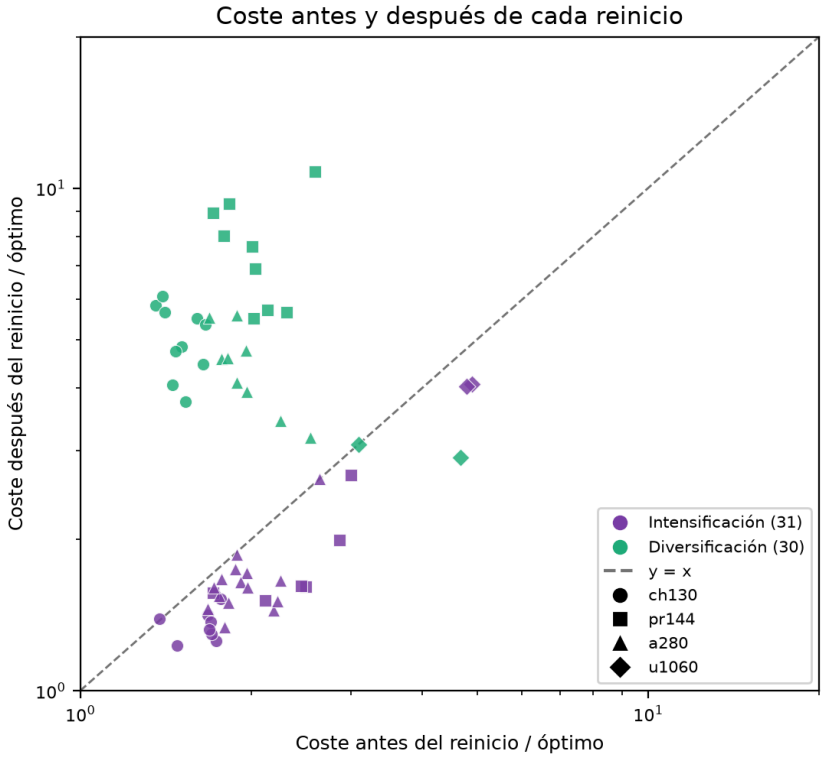
La búsqueda tabú es el mejor algoritmo en todas las instancias en las que dispone de presupuesto para salir del primer óptimo local. Su desviación media es del 20,76 % en ch130, 51,73 % en pr144, 37,28 % en a280 y 139,81 % en u1060, frente al 91,85–375,29 % de la búsqueda local partiendo de la misma solución. Reduce el coste del óptimo local de la BL entre un 37,05 % y un 49,61 % de media, y su desviación típica es entre 2,19 y 2,90 veces menor que la de la BL en ch130, pr144 y a280. Es, por tanto, también más robusta frente a la semilla.

Para entender el origen de la mejora hay que analizar cómo evoluciona cada ejecución. La primera fase LS reproduce exactamente la trayectoria de la búsqueda local (mismas iteraciones de convergencia en la Tabla 4), como cabe esperar al partir de la misma solución con la misma regla de exploración. A partir de ahí se observan tres mecanismos:

- **La fase TS por sí sola produce mejoras apreciables pero irregulares.** En ch130, antes del primer reinicio, el mejor del momento baja de 12 238,21 a 9 869,11 (−19,36 %) con la semilla 12345678, y de 11 237,36 a 9 900,27 (−11,90 %) con la semilla 23456781. Aceptar el mejor movimiento no tabú, aunque empeore, permite cruzar barreras de coste pequeñas, pero con el operador de intercambio y una tenencia de 10 la búsqueda tiende a oscilar alrededor del óptimo local.
- **Las intensificaciones son el mecanismo decisivo**. Al reconstruir la solución siguiendo los arcos más frecuentes, se combinan los arcos que han aparecido reiteradamente en los óptimos locales visitados, y el desempate por distancia completa el recorrido con un criterio de vecino más cercano. En ch130 con la semilla 23456781, la primera intensificación genera directamente una solución de 8 631,26, mejor que cualquiera de las encontradas hasta entonces (9 900,27). La Figura 3 confirma que este comportamiento es general: de los 31 reinicios por intensificación de todas las instancias y semillas, 29 quedan por debajo de la diagonal, es decir, producen una solución mejor que la de partida, con una reducción mediana del 17 %. Las dos excepciones (ch130, semilla 34567812, iteración 4 859, y a280, semilla 34567812, iteración 1 696) empeoran el coste menos de un 1 % (de 8 412,42 a 8 492,25 y de 6 805,31 a 6 809,12). Además, en las nueve ejecuciones de ch130, pr144 y a280, la mejor solución global aparece entre 10 y 58 iteraciones después de una intensificación: en ocho de ellas, durante la fase LS que pule la solución reconstruida (por ejemplo, ch130 con semilla 34567812: intensificación en la iteración 8 354 y mejor global en la 8 369), y en la restante (a280, semilla 23456781), en las primeras iteraciones de la fase TS que sigue.
- **Las diversificaciones generan soluciones peores** (de 22 930,22 a 37 194,17 en ch130), porque los arcos menos usados tras miles de iteraciones son precisamente los largos. En la Figura 3, las diversificaciones de las instancias pequeñas quedan todas muy por encima de la diagonal: multiplican el coste entre 1,3 y 5,2 veces (unas 3 veces en mediana). Su función es alimentar la MLP con zonas no exploradas: en ninguna instancia pequeña la mejor solución procede directamente de una diversificación, pero, al incorporar a la MLP arcos nuevos, cabe esperar que las intensificaciones siguientes dispongan de estadísticas menos sesgadas hacia la primera cuenca explorada.

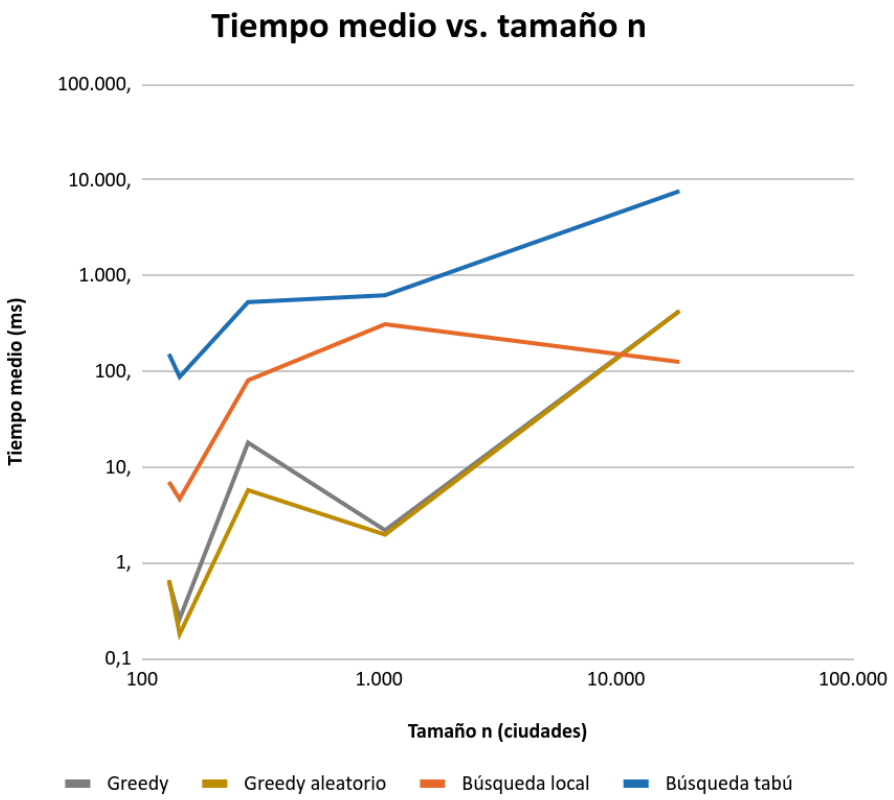
Un caso singular es u1060. Allí la primera fase LS consume entre 7 395 y 7 518 iteraciones (el 73,95–75,18 % del presupuesto) y solo hay 1 o 2 reinicios. Con tan poca historia, la mayoría de los arcos cortos tienen todavía frecuencia cero, de modo que incluso la diversificación equivale casi a un vecino más cercano. En la semilla 12345678, el reinicio por diversificación produce una solución de 652 105,91, ya un 36,92 % mejor que el óptimo local de la BL (1 033 783,80). Es una de las dos diversificaciones, de las 30 realizadas, que mejoran el coste, y aparece en la Figura 3 como el punto verde claramente por debajo de la diagonal; la otra (semilla 34567812, iteración 9 708) solo lo reduce un 0,13 % y queda sobre la propia diagonal. La fase LS posterior lleva esa solución a 388 285,65. En las otras dos semillas el reinicio es una intensificación, que también mejora el coste (a 913 120,11 y 904 170,95), pero mucho menos: los arcos más frecuentes son todavía los de la trayectoria de la BL, heredados en gran parte de la solución inicial. La mejor solución global de las tres semillas de u1060 se encuentra en las iteraciones 9 526–10 000, lo que indica que la búsqueda seguía mejorando cuando se agotó el presupuesto; con más iteraciones la diferencia con la BL sería mayor. La gran desviación típica de la tabú en u1060 (129 818,10) refleja precisamente la calidad de la solución reconstruida en ese primer reinicio.

En **d18512** la búsqueda tabú no llega a ejecutar ni una iteración de la fase TS: la fase LS agota las 10 000 iteraciones antes del óptimo local, y el resultado es idéntico al de la búsqueda local. Un presupuesto fijo de iteraciones, independiente de n, no es adecuado para instancias de este tamaño.



«Figura 3. Coste de la solución antes y después de cada reinicio de la búsqueda tabú (75 reinicios: todas las instancias y semillas), dividido por el óptimo conocido para poder comparar instancias. Ambos ejes en escala logarítmica. Los puntos por debajo de la diagonal y = x corresponden a reinicios que mejoran la solución; los puntos por encima, a reinicios que la empeoran. El coste «después» es el de la solución reconstruida, antes de la fase LS que la mejora a continuación. »

3.4 Tiempos, desviaciones e influencia del tamaño



«Figura 4. Tiempo medio de ejecución frente al tamaño de la instancia (media de las tres semillas; el Greedy es una única ejecución), con ambos ejes en escala logarítmica. En d18512 la búsqueda local se detiene en el límite de 10 000 iteraciones, mientras que la tabú obtiene el mismo coste con un tiempo 165 veces mayor. »

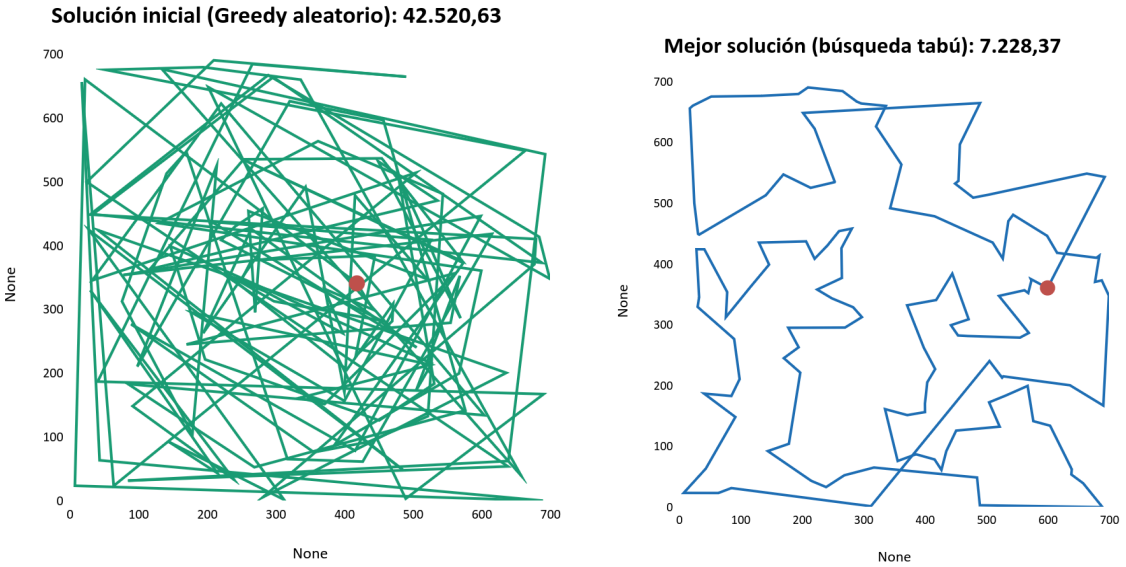
- **Greedy y Greedy aleatorio** son O(n²): ambos calculan la suma de distancias de cada ciudad al resto y, en el aleatorio, cada paso elimina la ciudad elegida de la lista ordenada, con coste O(n). De media tardan menos de 20 ms hasta u1060 (Figura 4). En d18512 ambos tardan prácticamente lo mismo (425,89 ms de media el Greedy aleatorio frente a 425,75 ms el Greedy): el tiempo lo domina el cálculo de las sumas de distancias, y la elección aleatoria entre las K primeras ciudades apenas añade coste.
- **Búsqueda local.** Cada ciudad no marcada explora hasta n − 1 vecinos, así que la comprobación final de optimalidad cuesta al menos O(n²) evaluaciones. El tiempo pasa de 6,95 ms en ch130 y 4,62 ms en pr144 a 80,49 ms en a280 y 309,99 ms en u1060: un crecimiento superlineal coherente con O(n · iteraciones), donde además las iteraciones crecen con n. d18512 rompe la tendencia (125,14 ms, la caída de la curva naranja en la Figura 4). La búsqueda se corta en 10 000 iteraciones antes de llegar a la comprobación final de optimalidad, y en una solución tan mala cada mejora se encuentra entre los primeros vecinos explorados.

- **Búsqueda tabú.** Con un presupuesto fijo de 10 000 iteraciones y un coste O(n) por iteración (explorar todos los intercambios de una ciudad, más actualizar los n arcos en la MLP), cabría esperar un tiempo aproximadamente lineal en n: 150,36 ms en ch130, 526,74 ms en a280 y 620,33 ms en u1060. En la Figura 4, sin embargo, la curva no es una recta: entre a280 y u1060 el tiempo apenas crece, porque en u1060 casi el 90 % de las iteraciones pertenecen a la primera fase LS, más barata que la fase TS (la BL completa tarda 309,99 ms). Es entre 6,54 y 21,64 veces más lenta que la BL en las instancias pequeñas, un coste moderado para una reducción del coste del 37,05–49,61 %. En d18512, aun siguiendo la misma trayectoria que la BL, tarda 7 604,12 ms frente a 125,14 ms. La diferencia se debe a la reserva de la matriz de memorias (685,39 MB) y, sobre todo, a la actualización de la MLP: 10 000 · 18 512 ≈ $1{,}85 \cdot 10^8$ incrementos en posiciones dispersas de esa matriz, con fallos de caché continuos. Este tiempo es, de hecho, del orden del que predice la tendencia lineal: extrapolando desde u1060 (620,33 ms · 18 512 / 1 060 ≈ 10 800 ms) se obtiene un valor del mismo orden que el medido. Lo excepcional en d18512 no es la lentitud de la tabú, sino la rapidez de la búsqueda local truncada.
- **Desviaciones e influencia del tamaño.** La desviación de todos los algoritmos crece de forma marcada con n (Tabla 1). En los constructivos se debe a que el coste de un recorrido desordenado crece como $n \cdot \bar{d}$, mientras que el óptimo euclídeo crece aproximadamente como $\sqrt{(n \cdot A)}$, donde A es el área de la región. En las metaheurísticas se añade que el presupuesto de 10 000 iteraciones no escala: es 76,92 veces n en ch130, pero solo 9,43 veces n en u1060 y 0,54 veces n en d18512. Las instancias ch130, pr144 y a280 tienen un tamaño parecido y desviaciones del mismo orden. Las diferencias entre ellas responden más a la distribución espacial de cada instancia que a su tamaño: pr144 es la peor de las tres para la tabú (51,73 %) pese a tener solo 14 ciudades más que ch130.

3.5 Análisis de la mejor solución obtenida

7228,37	18,30 %	1262 → 4
Coste — ch130, búsqueda tabú, semilla 34567812	Desviación sobre el óptimo conocido (6 110)	Cruces de arcos: solución inicial → mejor solución

La mejor solución en términos relativos se obtuvo en ch130 con la búsqueda tabú y la semilla 34567812. Su coste, 7 228,37, está un 18,30 % por encima del óptimo. Se partió de una solución del Greedy aleatorio de 42 520,63 (595,92 % de desviación) y se redujo el coste un 83,00 %. La trayectoria fue la siguiente: la primera fase LS converge en la iteración 469 con un coste de 11 690,79; la fase TS lo reduce hasta 10 575,87 antes de estancarse; la primera intensificación, en la iteración 2 508, construye una solución de 9 296,99 a partir de la MLP, que la fase LS lleva a 8 226,49. Tras otros seis reinicios (tres intensificaciones y tres diversificaciones), en los que el mejor global baja a 7 435,38 después de la intensificación de la iteración 4 326, la intensificación de la iteración 8 354 construye una solución de 7 507,54, y solo 15 iteraciones después la fase LS alcanza el mejor global. Las tres mejoras del mejor global siguen, por tanto, a una intensificación. El único reinicio posterior (una diversificación) no lo supera, pero el mejor global aparece en la iteración 8 369, de modo que con más presupuesto la búsqueda podría haber seguido mejorando.



«Figura 5. Recorridos en ch130 con la semilla 23456781. A la izquierda, la solución inicial del Greedy aleatorio, con su característico patrón de arcos radiales en torno a la ciudad de partida (marcada con un círculo). A la derecha, la mejor solución de la búsqueda tabú.»

Significado teórico. En el TSP euclídeo, todo recorrido óptimo carece de cruces: si dos arcos se cortan, intercambiar sus extremos (un movimiento 2-opt) acorta estrictamente el circuito por la desigualdad triangular. La solución inicial tiene 1 262 cruces y la mejor solución conserva 4. Aunque visualmente es casi un recorrido «limpio», esos cuatro cruces prueban que la solución no es ni siquiera 2-óptima y que un vecindario 2-opt la mejoraría de inmediato. Parte del 18,30 % de desviación restante se debe a esos cruces y a órdenes de visita locales subóptimos, que el operador de intercambio no puede corregir sin pasar por soluciones intermedias peores.

Significado práctico. Para UJACAR, esta ruta supone recorrer un 18,30 % más de distancia que la ruta óptima, frente al 83,92 % de la mejor búsqueda local y al 595,92 % del Greedy aleatorio. Se obtiene en 137,30 ms, un tiempo perfectamente compatible con replanificar la ruta cada día. Para tamaños realistas de reparto (100–300 paradas), la búsqueda tabú da soluciones utilizables. La mejora más rentable sería sustituir el operador de intercambio por 2-opt y partir de una solución inicial construida por vecino más cercano, eligiendo cada ciudad entre las más próximas a la última añadida en lugar de por su centralidad, dos cambios que atacan directamente las dos limitaciones identificadas en este análisis.

3.6 Ficheros de registro (log)

Enlace al directorio de Google Drive con los ficheros de registro:
https://drive.google.com/drive/folders/1hp2j4qpHGmkReyrqbk0d7n9f0Vta8d-S?usp=drive_link